

CSc 453 — Assignment 6

Christian Collberg
University of Arizona, Department of Computer Science

April 1, 2026

1 Introduction

For this assignment you will be given a Luca compiler that generates a typed byte code for a stack machine. We will call this bytecode the LVM. Your job is to write an interpreter that reads and executes LVM programs.

2 Getting Started

This assignment can be done in teams of one or two.

To get started, follow these instructions:

1. Unzip `assignment-6.zip` which gives you these files:

```
assignment-6/assignment-6.pdf
assignment-6/firstname_lastname/tests/*.luc
assignment-6/firstname_lastname/vm_files/*.out
assignment-6/lucac_macos_m1
assignment-6/lucac_linux_x86_64
assignment-6/firstname_lastname/collaboration.json
assignment-6/firstname_lastname/survey.json
assignment-6/firstname_lastname/luca_run_switch.c
assignment-6/firstname_lastname/luca_run_indirect.c
survey
```

2. Then, assuming your name is Pat Jones, execute these commands:

```
> cd assignment-6
> mv firstname_lastname pat_jones
```

If you are working in a group of two, pick one of your names.

3. First write your switch-threaded interpreter in `luca_run_switch.c` and then your indirect threaded interpreter in `luca_run_indirect.c`.

3 Implementation

Your task is to write an interpreter for the language LUCA.

1. You should actually write 2 versions of the interpreter, one that uses *switch dispatch* and one that is implemented using *indirect threaded dispatch*.

2. The interpreters should be named `luca_run_switch` and `luca_run_indirect`. You first compile your program to LVM code using the provided Luca front-ends:

```
> lucac_macos_m1      -i x.luc -v x.vm
> lucac_linux_x86_64 -i x.luc -v x.vm
```

And then you run your interpreters on that vm code:

```
> luca_run_switch x.vm
> luca_run_indirect x.vm
```

3. **This assignment should be coded in C. For the indirect threaded dispatch you should use gcc C's labels-as-values.**
4. For this assignment you don't have to handle procedures. That will be the subject of the next assignment.

3.1 Runtime Errors

There are two errors you need to check for at runtime:

1. Arrays are indexed from 0; that is, an array declared as `ARRAY 100 OF INTEGER` has the index range `[0..99]`. It is a checked run-time error to go outside these index bounds. You should generate the following error message:

```
<RUNTIME_ERROR pos="3" message="Array index out of range"/>
```

Note that the source code line number is part of the error message.

2. Division by zero should generate this error message:

```
<RUNTIME_ERROR pos="3" message="Division by zero"/>
```

Note that this applies to the mod operator too.

4 LVM Bytecode File

Consider this program:

```
PROGRAM P;

VAR I : INTEGER;
VAR J : INTEGER;
BEGIN
  I := 0;
  J := 0;
  WRITE J+I; WRITELN;
END.
```

This is the corresponding LVM file:

```

55 16    # [0] 0: DEF_GLOBALS
1        # [1] 0: PROG_BEGIN
5 0      # [2] 1: PUSHADDR_global
18 0     # [3] 10: PUSHCONST_i
20       # [4] 19: STORE_i
5 8      # [5] 20: PUSHADDR_global
18 0     # [6] 29: PUSHCONST_i
20       # [7] 38: STORE_i
5 8      # [8] 39: PUSHADDR_global
19       # [9] 48: LOAD_i
5 0      # [10] 49: PUSHADDR_global
19       # [11] 58: LOAD_i
8        # [12] 59: PLUS_i
44       # [13] 60: WRITE_i
51       # [14] 61: WRITELN
2        # [15] 62: PROG_END

```

Anything following the `#`-character is a comment. We will not use this information in our interpreter; it's just there to help us understand the bytecode.

The first number on each line is the opcode. 5, for example, represents `PUSHADDR_global`, and 18 `PUSHCONST_i`. Opcodes are one byte long. All following arguments are 8 bytes long.

Here's a piece of the interpreter declaration:

```

void interpret(char prog[]) {
    ...
    while(1) {
        int op = prog[pc];
        switch (op) {
            case PROG_BEGIN: {
                pc += PROG_BEGIN_SIZE;
                break;
            }
        }
    }
}

```

Consider this part of the input:

```

20       # [7] 38: STORE_i
5 8      # [8] 39: PUSHADDR_global
19       # [9] 48: LOAD_i
5 0      # [10] 49: PUSHADDR_global
19       # [11] 58: LOAD_i
8        # [12] 59: PLUS_i
44       # [13] 60: WRITE_i
51       # [14] 61: WRITELN

```

What will the `prog` array look like for this piece of the input? Well, it's an array of bytes, like this:

38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61
20	5	0	0	0	0	0	0	0	8	19	5	0	0	0	0	0	0	0	0	19	8	44	51

The first byte is 20, which is the opcode for `STORE_i`. The next byte is 5, the opcode for `PUSHADDR_global`. We can learn all this from the definitions of the bytecode instructions in Section 5. Here's a small excerpt with the relevant instructions:

Instruction	Opcode	Arguments	Stack Pre	Stack Post	Description
PUSHADDR_global	5	Offset	[]	[A]	Push the address A of the formal parameter at given offset.
PLUS_i	8		[A, B]	[A + B]	
LOAD_i	19		[A]	[*A]	Push the value pointed to by the pointer on top of the stack.
STORE_i	20		[A, B]	[]	Store B at address A .
WRITE_i	44		[A]	[]	Write the integer A to standard output.
WRITELN	51				Write a newline.

We see that `PUSHADDR_global` takes an integer argument, 8 in our case. The binary representation of this 8 is in bytes 40-47.

Programs start executing at the `PROG_BEGIN` instruction.

5 The Luca Virtual Machine (LVM)

Opcodes that end in `_i` work on integers, those that end in `_f` on reals.

For simplicity, all basic types (`INTEGER`, `REAL`, `CHAR`, `BOOLEAN`) are 8 bytes wide. I.e., `INTEGER` and `REAL` correspond to `long` and `double`, respectively, in C on *lectura*.

Instruction	Opcode	Arguments	Stack Pre	Stack Post	Description
DEF_STRING	0	number string			Declare string.
DEF_GLOBALS	55	globalSz			Size of global variables, in bytes.
PROG_BEGIN	1				Execution starts here.
PROG_END	2				Execution ends here.
PUSHADDR_local	3	Offset	[]	[A]	Push the address A of the local variable at given offset.
PUSHADDR_formal	4	Offset	[]	[A]	Push the address A of the formal parameter at given offset.
PUSHADDR_global	5	Offset	[]	[A]	Push the address A of the formal parameter at given offset.
FIELD	6	Offset	[R]	[R + Offset]	Compute the address of the field at <code>Offset</code> in the record whose base address is B .
INDEX	7	elSz elCount	[B, I]	[B + elSz * I]	From the array base address B and index value I compute the address of the array element.
PLUS_i	8		[A, B]	[A + B]	
MINUS_i	9		[A, B]	[A - B]	
MULT_i	10		[A, B]	[A * B]	
DIV_i	11		[A, B]	[A / B]	
MOD_i	56		[A, B]	[A % B]	

EQ_i	12	Offset	$[A, B]$	$[\]$	If $A == B$ goto PC+Offset, else fall through.
GE_i	13	Offset	$[A, B]$	$[\]$	If $A \geq B$ goto PC+Offset, else fall through.
GT_i	14	Offset	$[A, B]$	$[\]$	If $A > B$ goto PC+Offset, else fall through.
LT_i	15	Offset	$[A, B]$	$[\]$	If $A < B$ goto PC+Offset, else fall through.
LE_i	16	Offset	$[A, B]$	$[\]$	If $A \leq B$ goto PC+Offset, else fall through.
NE_i	17	Offset	$[A, B]$	$[\]$	If $A \neq B$ goto PC+Offset, else fall through.
PUSHCONST_i	18	Value		[Value]	
LOAD_i	19		$[A]$	$[*A]$	Push the value pointed to by the pointer on top of the stack.
STORE_i	20		$[A, B]$	$[\]$	Store B at address A .
UMINUS_i	21		$[A]$	$[-A]$	
PLUS_f	22		$[A, B]$	$[A + B]$	
MINUS_f	23		$[A, B]$	$[A + B]$	
MULT_f	24		$[A, B]$	$[A + B]$	
DIV_f	25		$[A, B]$	$[A + B]$	
EQ_f	26	Offset	$[A, B]$	$[\]$	If $A == B$ goto PC+Offset, else fall through.
GE_f	27	Offset	$[A, B]$	$[\]$	If $A \geq B$ goto PC+Offset, else fall through.
GT_f	28	Offset	$[A, B]$	$[\]$	If $A > B$ goto PC+Offset, else fall through.
LT_f	29	Offset	$[A, B]$	$[\]$	If $A < B$ goto PC+Offset, else fall through.
LE_f	30	Offset	$[A, B]$	$[\]$	If $A \leq B$ goto PC+Offset, else fall through.
NE_f	31	Offset	$[A, B]$	$[\]$	If $A \neq B$ goto PC+Offset, else fall through.
PUSHCONST_f	32	Value	$[\]$	[Value]	
LOAD_f	33		$[A]$	$[*A]$	Push the value pointed to by the pointer on top of the stack.
STORE_f	34		$[A, B]$	$[\]$	Store B at address A .
UMINUS_f	35		$[A]$	$[-A]$	
TRUNC	36		$[A]$	$[B]$	Truncate the floating point value A to an int B .
FLOAT	37		$[A]$	$[B]$	Convert the integer A to a float B .
JUMP	41	Addr			Goto PC+Offset.
WRITE_i	44		$[A]$	$[\]$	Write the integer A to standard output.
WRITE_f	45		$[A]$	$[\]$	Write the float A to standard output.
WRITE_c	46		$[A]$	$[\]$	Write A as a character to standard output.
WRITE_s	47		$[A]$	$[\]$	Write the string whose address is A to standard output.

WRITELN	51				Write a newline.
READ_i	48		[A]	[]	Read an integer from standard input and store at address <i>A</i> .
READ_f	49		[A]	[]	Read a float from standard input and store at address <i>A</i> .
READ_c	50		[A]	[]	Read a character from standard input and store at address <i>A</i> .

6 Academic Integrity

- Don't show your code to anyone outside your team.
- Don't read anyone else's code.
- Don't discuss the details of your code with anyone.
- If you need help with the assignment see the TAs or the instructor.

6.1 Use of AI

You must write all code by hand without the use of any AI tools.

Q: Can I upload the assignment to an AI tool to explain it to me?

A: No. If you don't understand the assignment, ask the TAs.

Q: Can I use an AI tool to generate test cases?

A: No.

Q: In fact, are there *any* possible use cases for which it is OK for me to use AI tools in this assignment?

A: No, there are *no* such use cases.

Q: I will be using AI coding tools in my future job. So why can't I prepare for that by using such tools in this class?

A: This is a compiler class. The goal is to learn about compilers. If you want to learn about AI, take an AI class.

Q: But you will never be able to tell if my code was generated by an AI coding tool!

A: ROTFL!

7 Submission

To submit, follow these instructions:

1. Verify that your submission contains a working **Makefile** and *all* the files necessary to build the interpreters. Your program must compile out of the box. The graders will *not* try to debug your program or your makefile for you.
2. You should test the interpreters on *lectura*.
3. Fill out `collaboration.json` with information about your group.
4. Optionally fill out `survey.json` to give me feedback about the assignment.
5. Package up your answers and verify that they look OK:

```
> zip -r pat_jones.zip pat_jones
> assignment-6/validator.sh pat_jones.zip
==== This is the LigerLabs assignment validator ====
...
==== 6: Checking that all problems have been answered.
==== 7: Checking that all answer files have valid structure.
```

6. Upload `pat_jones.zip` to d2l. If you are working in a group of two, you should only submit once.

8 Grading

70 test cases have been provided for you in `assignment-6/tests/*/*.luc`. The corresponding vm files are in `vm_files/*/*.vm`.

Each test case will give you 0 points if you get it wrong, 0.6 point if you get it right. You get points for both interpreters. In other words, if your interpreter is perfect, you can get $2 * 0.6 * 70 = 84$ points.

You can get an additional 16 points from secret test cases.

When we run the test cases we will remove everything after the `#`-character to make sure that you don't make use of the information in the comments in your interpreter.

A Optional Extensions

If you want to challenge yourself some more add one or more of these extensions. Submit extensive test cases for each extension and a `README` file that describes what you did. There is no extra credit, just the satisfaction of a job well done!

A.1 TOS

Implement the top-of-stack optimization. Run some experiments to see if this optimization is worthwhile or not.

Read this paper: <https://dl.acm.org/doi/pdf/10.1145/207110.207165>.

A.2 Dispatch

Implement all dispatch methods and time them to see which ones work best.